ClearSafe California – Defensible Space Agent (Version One)

Course Project Report (Agentic AI / Systems)

March 2026

Repository	DefensibleSpaceAgent
Runtime	Python 3 + Flask
Entry point	web_app.py
Core agent	src/agent.py (planner/validators/executors)

Abstract

ClearSafe California is a structured, tiered agent prototype for California-only wildfire defensible-space and property wildfire context assessments. The system separates an LLM-produced execution specification (planner) from deterministic validation and tool execution, and it degrades gracefully when external services are missing. Version One implements two user-facing tiers: a Baseline (free) address-level overview and a Full (paid) property-focused assessment that can incorporate remote-sensing vegetation signals (NDVI) when Google Earth Engine is configured. This report documents the current architecture, multi-LLM orchestration, tool integrations, and safety/validation strategy, and it discusses limitations and failure modes grounded in the repository's implemented behavior.

1. Project Overview and Goals

Wildfire risk and defensible-space planning are high-impact, safety-relevant tasks where users often seek actionable guidance but face information gaps, inconsistent sources, and location-specific constraints. This project targets a narrow, well-scoped problem: producing a California-only structured wildfire/defensible-space assessment artifact from minimal user input (address and/or coordinates), while being explicit about uncertainty and the limits of remote evidence.

The core goals in Version One are:

- Provide a tiered workflow that clearly separates coarse baseline context from more detailed (optional) property-centered signals.
- Use an LLM to generate a machine-readable plan (execution spec) that is validated and executed by deterministic code.
- Enforce scope (California-only) and constrain behavior via whitelists and validation invariants, with conservative fallbacks when LLMs or tools are unavailable.

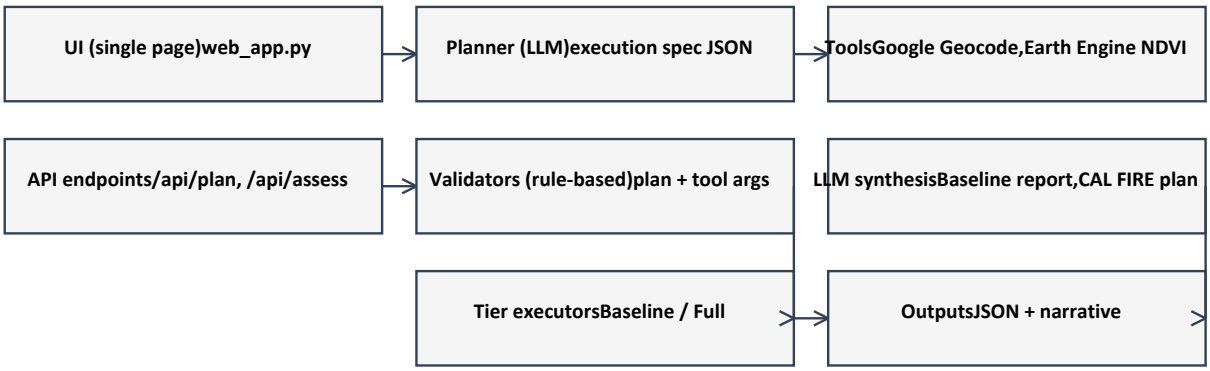
Intended users are homeowners and reviewers who want a clear demonstration of agent orchestration, validation, and tool integration in a socially important domain, without overclaiming correctness or official status.

2. Chosen Domain

This project best fits "other approved domain": a geospatial, safety-critical decision-support agent for wildfire defensible-space planning (California-focused). It is not primarily a database agent, website-development agent, or game-building agent; the web UI is an interface to an agent pipeline whose domain logic centers on location validation, evidence gathering, and constrained recommendation synthesis.

3. System / Agent Architecture

Figure 1. Implemented system architecture (Version One).



3.1 High-level components (as implemented)

The Version One system is a Flask web application (`web_app.py`) plus a structured agent core (`src/`) with clear stage boundaries:

- Frontend/UI: Single-page HTML/JS served by Flask (`GET /`). Uses Google Places Autocomplete to capture a formatted address and lat/lng when available.
- API layer: `POST /api/plan` (planner-only) and `POST /api/assess` (end-to-end execution).
- Planner (LLM, JSON): Produces an internal execution spec with tier classification, location strategy, steps, and constraints (`src/prompts.py`, `src/agent.py`).
- Validators (rule-based): Normalize and strictly validate planner output and tool-arguments before execution (`src/validators.py`).
- Executors:
- Baseline executor: Dedicated orchestrator that runs a registry of baseline tools and produces a structured Baseline report (`src/baseline_executor.py`, `src/baseline_tools.py`).
- Full executor: Step interpreter inside `src/agent.py` that executes internal tools (geocoding, NDVI, placeholders) and triggers recommendation/report synthesis.
- Report synthesis (LLM):
- Baseline synthesis: Generates a structured Baseline report JSON from tool outputs.
- Full recommendations: Generates a structured CAL FIRE–aligned plan JSON.
- Final narrative: Generates a concise homeowner-facing narrative from plan + execution evidence.

3.2 Architecture diagram (conceptual mapping)

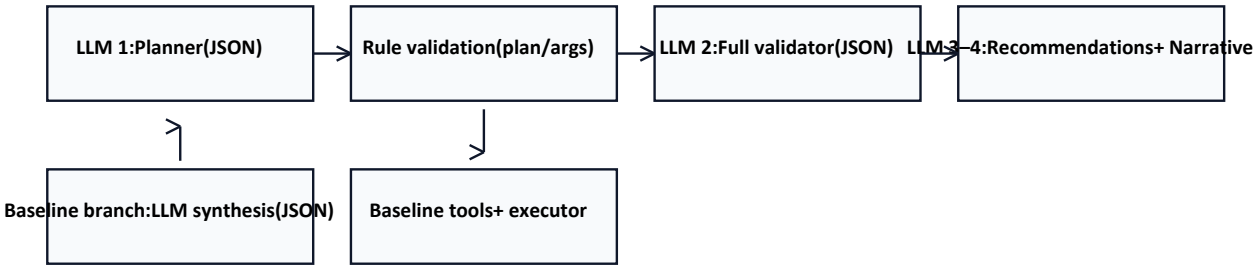
The implementation uses different names than the assignment's "planner/generator/validator/executor" terminology. The mapping in Version One is:

- Planner: `run_planner_only()` → execution spec JSON.
- Validator: `normalize_plan()` + `validate_plan()` + `validate_tool_args()` (rule-based) plus a small LLM validator step for Full tier.
- Executor: Baseline orchestrator (`execute_baseline_workflow`) and Full tier step executor loop.

- Generator/Reporter: Baseline synthesis and Full-tier recommendation + narrative generation (LLM), producing structured report objects and final text.

4. Multi-LLM Call Flow (Orchestration and Intermediate Representations)

Figure 2. Multi-LLM orchestration and branching (Baseline vs Full).



Version One coordinates multiple LLM calls through explicit intermediate representations and validation gates. The key intermediate artifacts are:

- Planner execution spec JSON (plan): request type (tier), assessment mode, location strategy, ordered steps, and constraints.
- Tool arguments dict (derived deterministically): address and NDVI parameters (buffer, date window, cloud threshold).
- Execution evidence dict: resolved coordinates, NDVI metadata (if available), fuel class, placeholders, and structured recommendation objects.
- Structured report JSONs:
- Baseline: `final_report` object with sections and evidence_used.
- Full: `calfire_recommendations` object with priority bands and zone plan.

4.1 Baseline (Free Tier) call flow

1. Planner LLM (JSON) produces a Baseline execution spec. 2. Rule-based validation ensures the plan is schema-correct, tier-consistent, and uses only allowed tools. 3. Baseline executor runs deterministic tools in a registry:

4. Baseline synthesis LLM (JSON) produces the structured Baseline report, with a deterministic fallback when the LLM is unavailable.

- resolve location (use provided coords or Google Geocoding),
- validate California scope (bounding box + optional geocode metadata),
- gather hazard/terrain/regional vegetation context (coarse placeholders),
- synthesize the Baseline report.

4.2 Full (Paid Tier) call flow

1. Planner LLM (JSON) produces a Full execution spec (baseline context + property steps + recommendation/report steps). 2. Rule-based validation enforces a strict tool whitelist, step ordering, constraint bounds, and tier requirements (e.g., the plan must end in `generate_full_report`). 3. Validator LLM (JSON) provides an additional "approval" gate for Full-tier execution (with a permissive fallback when not configured). 4. Full executor loop (deterministic) runs internal steps:

5. Narrative generator LLM (text) writes a concise homeowner-facing summary from the plan and execution evidence (with a deterministic fallback when not configured).

- resolve location (unless coordinates were provided),
- validate California scope (coarse bounding box backstop),
- gather hazard/terrain/vegetation context (coarse placeholders),
- compute NDVI via Google Earth Engine when available,
- classify fuel from NDVI (rule-based thresholds),
- run placeholders for slope/proximity/photo analysis,
- generate structured CAL FIRE–aligned recommendations.

5. Tools and APIs Used (Implemented)

5.1 OpenAI Chat Completions API

The system uses OpenAI's Chat Completions endpoint for:

- planner (JSON execution spec),
- Baseline synthesis (JSON report),
- Full recommendations (JSON mitigation plan),
- validator (JSON pass/fail report for Full tier),
- final narrative generation (text).

5.2 Google Geocoding API

When `GOOGLE_MAPS_KEY` is configured, the system calls Google's Geocoding API to convert an address to lat/lng and extract lightweight administrative metadata (state, county, city) used in California scope checks.

5.3 Google Earth Engine (Sentinel2 NDVI)

When `earthengine-api` is installed and Earth Engine is authenticated, the Full tier computes mean NDVI around the property coordinate using Sentinel2 surface reflectance imagery, with configurable date window and cloud threshold. The tool can also generate a thumbnail URL for visualization in the UI.

5.4 Internal "tools" and placeholders

Some analysis modules are implemented as explicit placeholders (returning "Not available in this build") to keep the execution spec stable while clearly distinguishing future work:

- property slope analysis,
- vegetation proximity/ring analysis,
- uploaded photo analysis.

6. Validation and Safety Checks

Version One's safety posture is primarily enforced through deterministic validation, explicit scoping, and conservative defaults:

- California-only scope enforcement:
- Baseline uses bounding-box checks plus (when available) geocode state metadata.
- Full includes a bounding-box backstop that blocks execution if coordinates appear outside California.
- Plan validation (`src/validators.py`):
- strict top-level schema (no extra keys),
- tool whitelist enforcement,
- tier-specific constraints (Baseline cannot include property-level tools; Full must include required tools and end with `generate_full_report`),
- dependency ordering (depends_on must reference earlier steps),
- constraint bounds (`buffer_m` in 1..500; `cloud_pct` in 0..100).
- Tool-argument validation:
- requires date window fields when NDVI is present in steps,
- requires an address only when geocoding is required.
- Graceful degradation:
- when OpenAI is not configured, LLM calls fall back to deterministic outputs (planner, baseline synthesis, recommendations, narrative),
- when Earth Engine is not configured, NDVI is returned as unavailable with structured reasons,
- when Google Geocoding is not configured and coordinates are not provided, location resolution fails cleanly (no fabricated coordinates).

These checks are designed to reduce the chance of executing unsupported steps, claiming unsupported evidence, or producing inconsistent multi-stage outputs.

7. Results and Discussion

7.1 What works well (current evidence)

- Structured planning + validation: A planner-produced execution spec is constrained by a strict schema and tool whitelist, allowing deterministic execution and clear "blocked" outcomes.
- Tier separation: Baseline and Full share the same planning/validation contracts but have distinct executors and outputs, reducing accidental cross-tier feature leakage.

- External-tool optionality: The system can still run in degraded mode without Earth Engine (Full continues with NDVI unavailable) and without OpenAI (deterministic fallbacks).
- Tested API shape and planning scenarios: The test suite checks planner scenarios, coordinate handling, and API response shape.

7.2 Limitations and failure modes (implemented)

- Coarse hazard/terrain/vegetation context: In both tiers, several “context” steps are placeholders and do not query authoritative hazard layers, DEM/slope models, land-cover datasets, or fire perimeters.
- California scope is coarse: Bounding-box validation is a safety backstop, not a jurisdictional boundary check.
- NDVI availability is environment-dependent: Full-tier NDVI requires Earth Engine installation and authentication; otherwise NDVI is unavailable and fuel classification becomes “No Data.”
- Photo analysis and proximity/slope analysis are not implemented: These appear as explicit placeholders even if they appear in the plan for Full tier.
- Documentation drift risk: Some legacy documentation describes an LLM “generator” step for tool-argument synthesis that is not present in the current implementation (tool args are derived deterministically).

7.3 Possible extensions (future work, not implemented)

Reasonable future extensions suggested by the repository’s current structure include:

- authoritative hazard layers (e.g., official map products) and recent fire perimeter proximity (with careful licensing and uncertainty handling),
- real slope/aspect computation via DEMs,
- vegetation proximity/ring metrics using parcel boundaries or buffered geometries,
- photo ingestion and vision-model analysis with explicit consent, privacy protections, and strong validation.

8. Conclusion

ClearSafe California (Version One) demonstrates a practical pattern for agentic systems in safety-relevant domains: use an LLM to propose a structured plan, enforce invariants with deterministic validators, execute only whitelisted steps with tool integration, and synthesize outputs with conservative fallbacks. The project’s strongest contribution is the explicit separation of planning, validation, execution, and reporting—combined with tiered scope control and clear “not implemented” placeholders that reduce overclaiming.

Appendix: Sample Interactions (not part of main page limit)

Figure A1. Sample interaction screenshot 1.

Figure A2. Sample interaction screenshot 2.

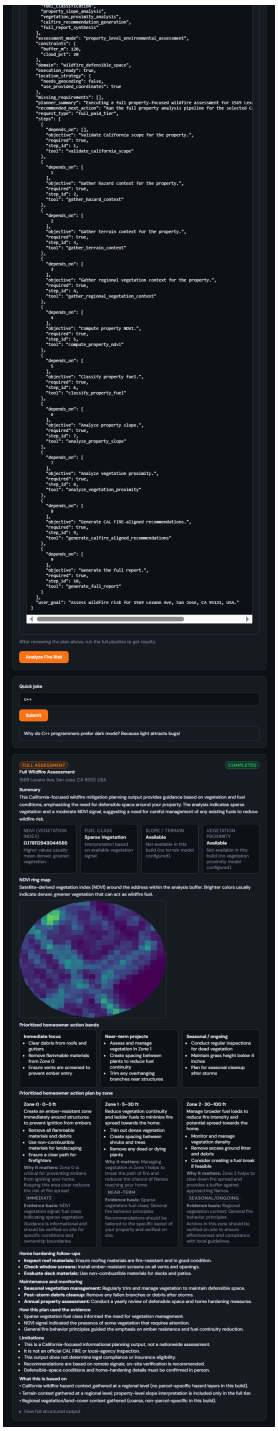



Figure A3. Sample interaction screenshot 3.